EL4001 Assignment Report #02

Canny Edge Detector and Harris Corner Detector

Jongwon Kim

September 21, 2026

1 Introduction

This assignment is about Canny edge detector and Harris corner detector. The provided codes are `canny.py` and `harris.py`.

For Canny, the code uses driving images from VPGNet dataset. There are six sequences with four weather labels: sunny (#S), cloudy (#C), rainy (#R), and night (#N). I used the first frame `000001.mat` of each sequence. The RGB image is `rgb_seg_vp[:,:,0:3]`.

```
img_b = cv2.GaussianBlur(img, (5, 5), 1.4) # kernel size & sigma
img_e = cv2.Canny(img_b, 50, 150) # lower & upper threshold
```

So the default Canny setting is kernel size 5×5, $\sigma = 1.4$, and thresholds $(50, 150)$. In this report I write Canny kernel size as "kernel", not k, because k is used later for the Harris constant. Q1 is to tune kernel size, σ, and the two thresholds for each of the six sets. Q2 is to discuss sunny, cloudy, rainy, and night from that result.

For Harris, the test image is `chess.jpg`.

```
dst = cv2.cornerHarris(gray, blockSize=5, ksize=5, k=0.04)
img[dst > 0.01 * dst.max()] = [255, 0, 255]
```

Default parameters are blockSize = 5, ksize = 5, and Harris constant $k = 0.04$. A pixel is marked if the response is larger than 1% of the maximum. Q3 is trying different `blockSize`, `ksize`, and k. Q4 is the limitation of Harris detector.

2 Q1. Canny Parameter Tuning

Canny has three parameters and they affect each other. Kernel size and σ are for Gaussian smoothing. If they are large, noise is reduced but thin lane marking can also disappear.

The two thresholds are hysteresis. Pixels with gradient larger than T_{high} are strong edges and they are always kept. Pixels between T_{low} and T_{high} are weak edges, and they are kept only if they are connected to a strong edge. So a high pair keeps only strong things like white paint or car boundary. A low pair can keep weak paint, but also keep road texture and rain drops if those weak pixels touch a strong edge.

I compared four settings for every sequence: the default $(5, 1.4, 50/150)$, a small blur with low threshold $(3, 0.8, 30/90)$, a strong blur with high threshold $(7, 2.0, 80/200)$, and the tuned values. The goal is to keep lane and road marking, not every small intensity change.

The chosen parameters are shown in Table 1. All three parameters (kernel size, σ, thresholds) are changed depending on the image.

For the cloudy intersection, default is already not bad. I only lower the thresholds a little to $(40, 120)$ so the zebra and the arrow is more clear.

Table 1: Tuned Canny parameters for the six driving sets. Default in the code is kernel 5×5, $\sigma = 1.4$, $(50, 150)$.

Sequence	Weather	kernel	σ	$(T_{\text{low}}, T_{\text{high}})$
20160503_1618_44	Cloudy	5	1.4	$(40, 120)$
20160512_1341_43	Sunny	5	1.2	$(70, 180)$
20160512_1410_30	Sunny	7	1.6	$(35, 110)$
20160518_2023_14	Night	7	2.0	$(20, 70)$
20160808_1346_45	Cloudy	5	1.6	$(35, 105)$
20160809_1433_15	Rainy	7	1.8	$(40, 120)$



Figure 1: Sunny city image (20160512_1341_43). From left: original, default $(5, 1.4, 50/150)$, small blur and low threshold, strong blur and high threshold, tuned $(5, 1.2, 70/180)$. Default still has many road texture. Low threshold is worse. Tuned result keeps the crosswalk better.

For the sunny city image, there is strong shadow and many texture on the road. If threshold is low, too many edges appear. So I raise the thresholds to $(70, 180)$ and keep kernel 5×5. σ is 1.2, a bit smaller than default, to keep the paint sharp.

The other sunny image is a tunnel exit. Even it has same #S label, half of the image is dark. If I use the high threshold like the city image, the yellow line in the shadow is gone. So this one needs larger kernel (7), larger σ (1.6), and lower thresholds $(35, 110)$. Same weather name does not mean same parameter.

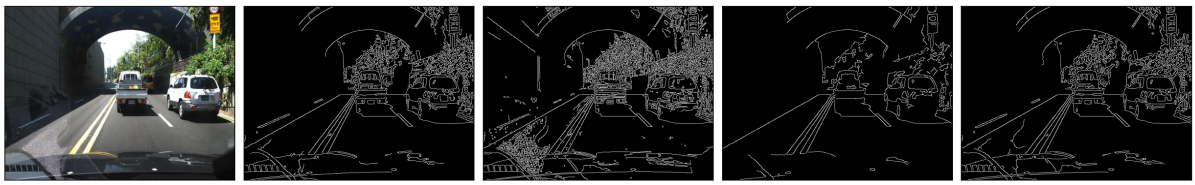


Figure 2: Sunny tunnel image (20160512_1410_30). Same order as Fig. 1. Tuned setting is kernel 7, $\sigma = 1.6$, $(35, 110)$. Compared with the city sunny image, this one needs more blur and lower thresholds because of the dark half.

Night image is the most different one. Default $(50, 150)$ is too high, so most dashed lane is missing. I use more blur (kernel 7, $\sigma = 2.0$) because of headlight and noise, and much lower thresholds $(20, 70)$.

The second cloudy set is a highway and the paint is weak. Default already miss the far dashed line. Thresholds $(35, 105)$ works better.

For rain, there is water drop on the windshield. Small kernel treat the drop as edges. Larger kernel 7 and $\sigma = 1.8$ reduce this. But if I also set high threshold like $(80, 200)$, the yellow lane also disappear. So rainy case is $(7, 1.8, 40/120)$.

From this, one parameter set cannot be used for all six images.

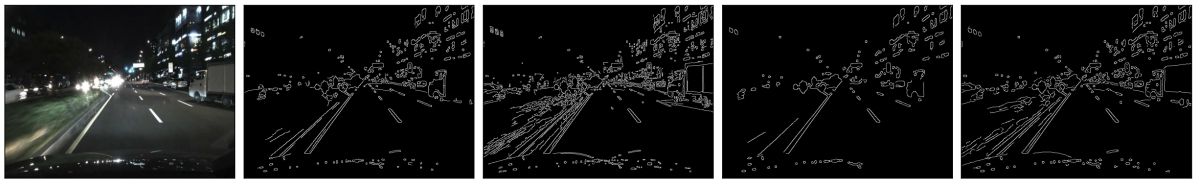


Figure 3: Night image (`20160518_2023_14`). Same order as Fig. 1. Default almost lose the dashed lane. Tuned setting kernel 7, $\sigma = 2.0$, $(20, 70)$ can keep it, but some headlight noise also remain.

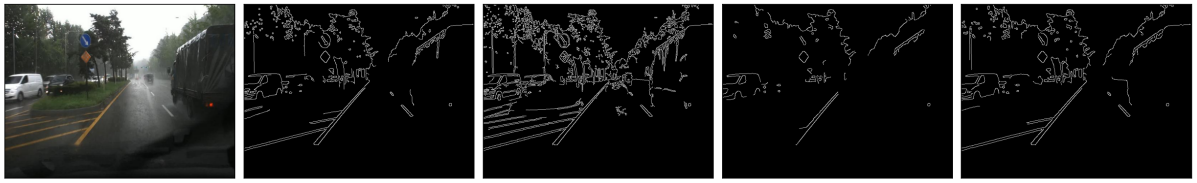


Figure 4: Rainy image (`20160809_1433_15`). Same order. The 4th panel (strong blur, high threshold) remove the yellow lane. Tuned result on the right keep the yellow line and reduce some speckle.

3 Q2. Weather and Lighting

Fig. 5 is one image for each weather after tuning. From top to bottom: sunny, cloudy, rainy, night. Left is original and right is Canny result.

Sunny. Contrast of white paint is high, but there is also dark shadow and rough asphalt. Canny find many edges that is not lane. So the main change is raising the thresholds. Kernel can stay small. But the tunnel image is also labeled sunny and it needs the opposite way, as shown in Fig. 2, because a large part is too dark.

Cloudy. Lighting is more flat and shadow is weak. Paint contrast is just medium. Kernel 5×5 is enough. I mainly lower the thresholds a little. For the hazy highway, default $(50, 150)$ already lose the far lane, so $(35, 105)$ is better.

Rainy. The problem is not only low contrast. Water drop and reflection make many small edges. So kernel and σ should be larger. Threshold should not be too high, otherwise the real lane is also removed. This is different from sunny case.

Night. The image is darkest and lane paint is weak next to strong headlight. Default threshold is too high. We need both more smoothing and much lower thresholds. Some false edges from the light still remain. I think this is hard to avoid only by Canny parameters.

In short, sunny usually needs higher threshold, cloudy needs a bit lower threshold, rainy needs more blur, and night needs more blur and much lower threshold. Also Canny threshold is about gradient size, not brightness. Night is dark, but the real reason to lower the threshold is that the lane-to-road gradient itself is smaller.

4 Q3. Harris Parameters

Harris uses the gradient around a pixel. From I_x and I_y we make the structure tensor M inside the `blockSize` window. Let λ_1 and λ_2 be the two eigenvalues of M. In the lecture, the Harris response is

$$R = \lambda_1 \lambda_2 - k\,(\lambda_1 + \lambda_2)^2. \tag{1}$$

If both eigenvalues are large, the pixel is a corner. If only one is large, it is more like an edge. If both are small, it is a flat region. `ksize` is the Sobel kernel for I_x and I_y. Here k is only the

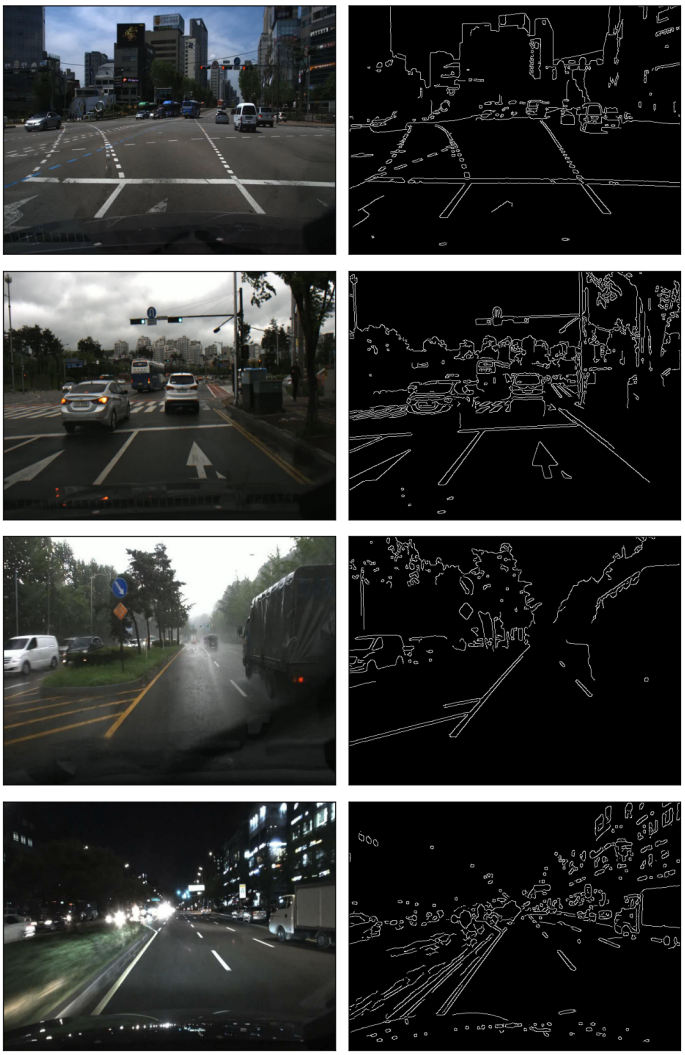


Figure 5: Four weather after tuning. Top to bottom: sunny, cloudy, rainy, night. Left: original. Right: Canny with the parameters in Table 1.

Harris constant, not the Canny kernel size. I compute R with this eigenvalue formula in the experiment code.

In Q3 I first count how many pixels have $R > 0.01\, R_{\max}$, same as the sample code. This number becomes large if each corner is a fat blob, even when the number of real corners is similar.

Fig. 6 is the default result. Magenta is the detected corner, same as the sample code. The chess intersections are found, but also the board frame, the letters, the glossy part, and even the hand.

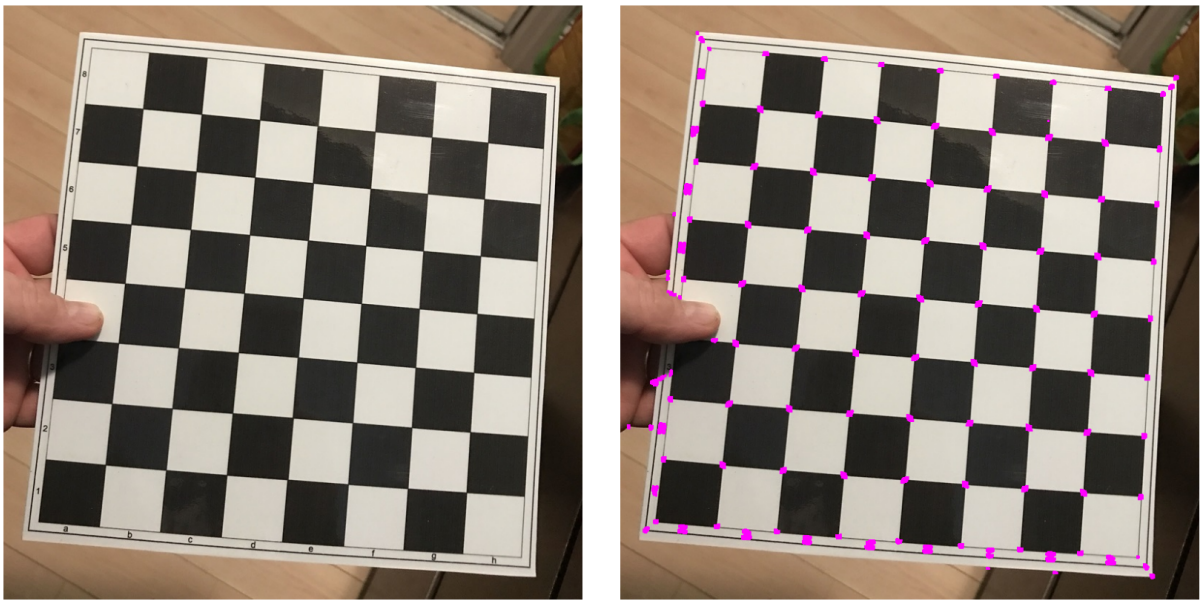


Figure 6: Left: `chess.jpg`. Right: Harris with default blockSize $= 5$, ksize $= 5$, $k = 0.04$.

4.1 blockSize

`blockSize` is the window size for M. Small window looks at very local area, so the mark is thin. Large window average more pixels, so each corner become a bigger blob, and some extra points on the frame also appear.

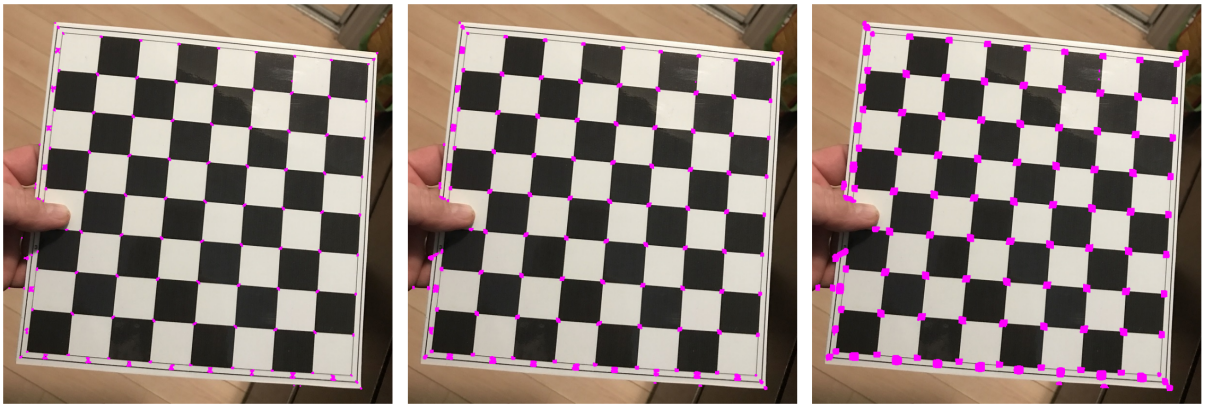


Figure 7: Different `blockSize`. Left: 2. Middle: 5 (default). Right: 11. `ksize` and k are fixed. Right image has much fatter marks.

The number of pixels over the threshold was 1639, 5650, and 18548 for 2, 5, 11. So larger `blockSize` mainly make the corner region bigger, not really much more independent corners.

Localization become worse.

4.2 ksize

`ksize` is Sobel aperture and it should be odd. I tried 1, 5, and 7 with blockSize = 5 and $k = 0.04$. On this chess board the change is not so big, because the corners are already very strong. A larger Sobel kernel just make the response a little more spread. The pixels over the threshold were 5184, 5650, and 6049. Compared with `blockSize` and k, this parameter has the smallest effect here, but the three results are still shown in Fig. 8.

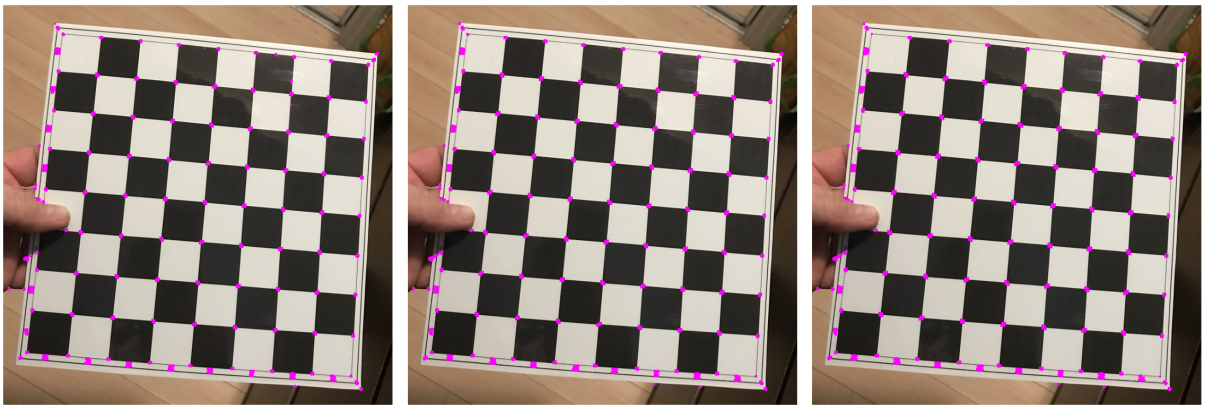


Figure 8: Different `ksize`. Left: 1. Middle: 5 (default). Right: 7. `blockSize`= 5 and Harris $k = 0.04$ are fixed. Interior chess corners stay in all three. The marks become a little more spread when `ksize` is larger.

4.3 Harris constant k

Harris k is used in $R = \lambda_1\lambda_2 - k(\lambda_1 + \lambda_2)^2$. If k is small, edge-like place can also have positive R, so more marks on the border. If k is large, the detector is more strict and the blobs become smaller.

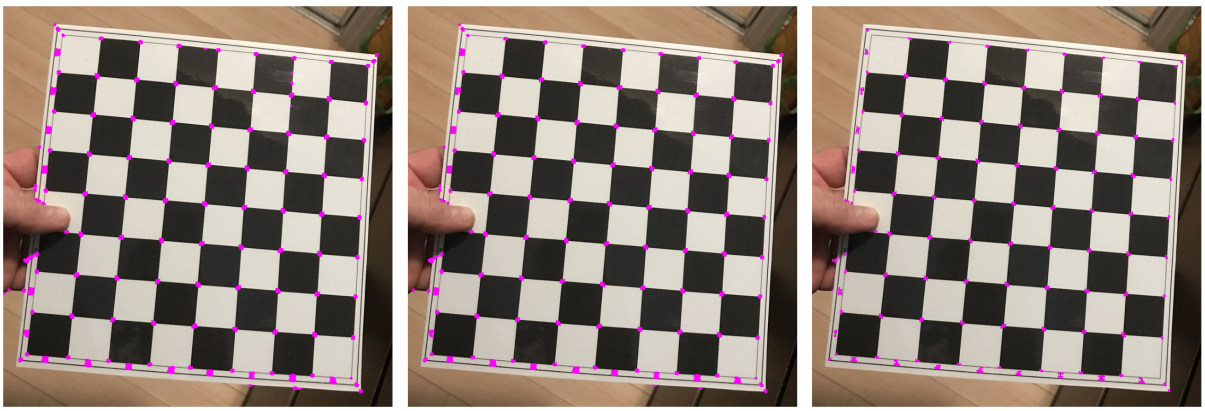


Figure 9: Different Harris k. Left: 0.01. Middle: 0.04 (default). Right: 0.20. `blockSize` and `ksize` are 5. Right one has smaller marks and less response on the frame.

Usually Harris k is around 0.04 to 0.06. Default 0.04 looks reasonable. $k = 0.20$ is too large for this image.

So, `blockSize` control the neighborhood size, `ksize` is smoothing of derivative, and Harris k is how much we punish edges.

5 Q4. Limitations of Harris

Harris is quite ok for rotation in the image plane, but it has several limitations. I tested with the same default parameters.

In this section I also count *peaks*. A peak is a pixel that is a 3×3 local maximum among the pixels with $R > 0.01\, R_{\max}$. This is not the same as the pixel count in Q3. Pixel count is large when the blob is fat. Peak count is closer to how many separate corner points we have.

First, it is sensitive to noise, because it use image gradient. When I add Gaussian noise, many fake corners appear. For strong noise ($\sigma = 40$), the peak number increase from about 241 to 3050. Also the threshold is 0.01 of the max value, so when the image is noisy the test still keep many small peaks.

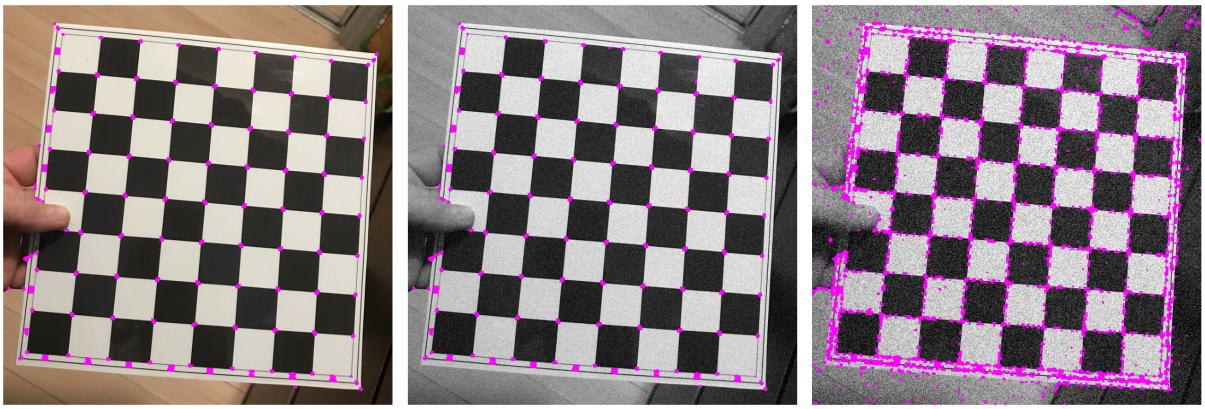


Figure 10: Noise test. Left: no noise. Middle: $\sigma = 15$. Right: $\sigma = 40$. Right image has a lot of false corners.

Second, Harris is not scale invariant. `blockSize` and `ksize` are fixed size. If I resize the chess image to 0.5 and 0.25 and run the same detector, some corners are missing and the remaining marks become very coarse. One window size cannot fit both large square and small square.

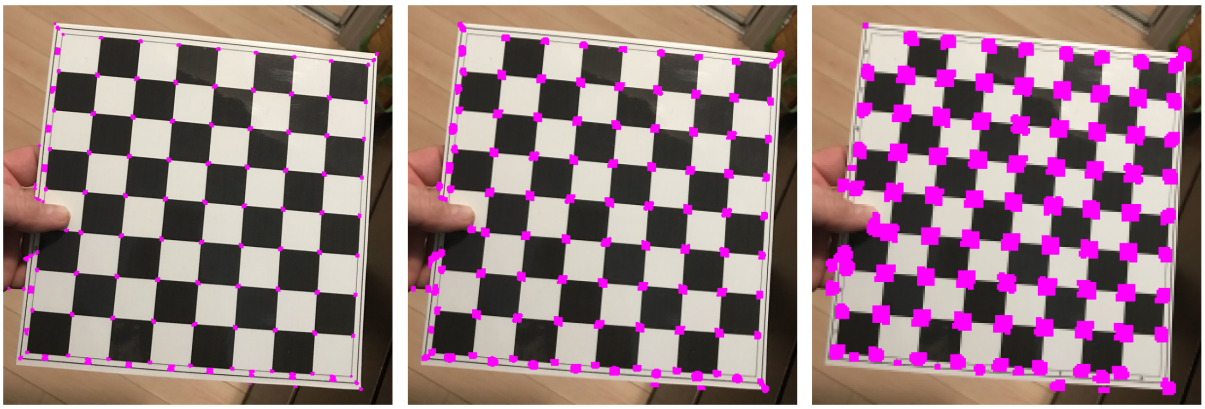


Figure 11: Scale test. Left: original size. Middle: 0.5×. Right: 0.25×. Detection is done at each scale, then the image is resized back only for showing. It is not the same corners with the same accuracy.

Blur is also a problem. Corner is a high frequency thing. After strong Gaussian blur the intersections become round, so the mark is not a point anymore. Because the threshold is relative, the corners do not all disappear, they just become thicker and the position is not accurate.

Affine or strong viewpoint change is another issue. After a shear transform, some intersections are gone (241 peaks to 187). Harris is not affine invariant. The original photo already has

some perspective and a bright highlight, and Harris also fire on that highlight which is not a real corner.

Finally, Harris does not know what is a useful corner. On the driving image, it respond to lane paint, cars, and building windows. Lane is more like an edge, but it still get many marks. On the chess image too, it detect the hand and the frame, not only the square corners.

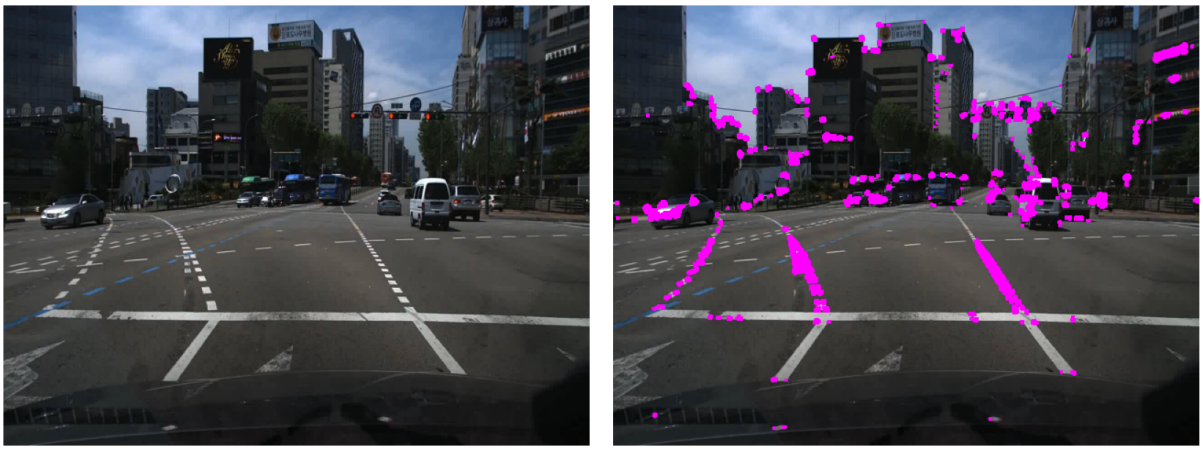


Figure 12: Left: a sunny driving frame. Right: default Harris. Many responses are on texture and lane lines, not only real corners.

Other small points: the 1% threshold depend on each image, so it is not a very stable rule. Also without non-maximum suppression, one corner is many pixels, not one point. And it uses only gray image, so color corner is ignored.

Overall, Harris is simple and works on the chess board, but it is not good when there is noise, scale change, blur, viewpoint change, or a lot of texture.

6 Summary

For Canny, I tuned kernel size, σ, and the two thresholds for six driving images. No single setting works for all. Sunny city image needs higher threshold. Cloudy is close to default but a bit lower threshold is better. Rain needs larger blur. Night needs larger blur and much lower threshold. The tunnel sunny image shows that even same weather label can need different parameters.

For Harris, `blockSize` changes how large each corner looks, `ksize` has smaller effect on this image, and Harris k changes how strict the detector is. Default $(5, 5, 0.04)$ is usable for `chess.jpg`.

The limitation of Harris is also clear from the experiment. It is sensitive to noise, not invariant to scale and affine transform, and it cannot tell a chess corner from a lane edge or a window. So it is a basic corner detector, but not enough for general scene like the driving image.